

Ans.1 import numpy as np
np1 = np.array([1,2,3,4])

np1
→ array([1,2,3,4])

import numpy as np

np2 = np.array([[1,2,3],[4,5,6],[7,8,9]])

np2
→ array([[1,2,3],
[4,5,6],
[7,8,9]])

Ans.2 d1 = {"roll no.": [1,2,3,4,5,6,7,8,9,10],

"marks": [90,80,70,60,50,40,30,20,10,100],

"attendance": [100,100,100,100,100,100,100,100,100,100],

"gender": ['M', 'M', 'F', 'M', 'M', 'M', 'F', 'M', 'F', 'M']}

df = pd.DataFrame(d1)

df

→ df["gender"].isnull().sum()

→ 0

→ df["marks"].mean()

→ 55.0

→ genders = df["gender"].value_counts()

print(df)

print(genders)

value_counts() → to count the occurrences of each gender using value_counts()

out
gender

M 6
F 4

Ans.3 import matplotlib.pyplot as plt

gender = ("m", "f")

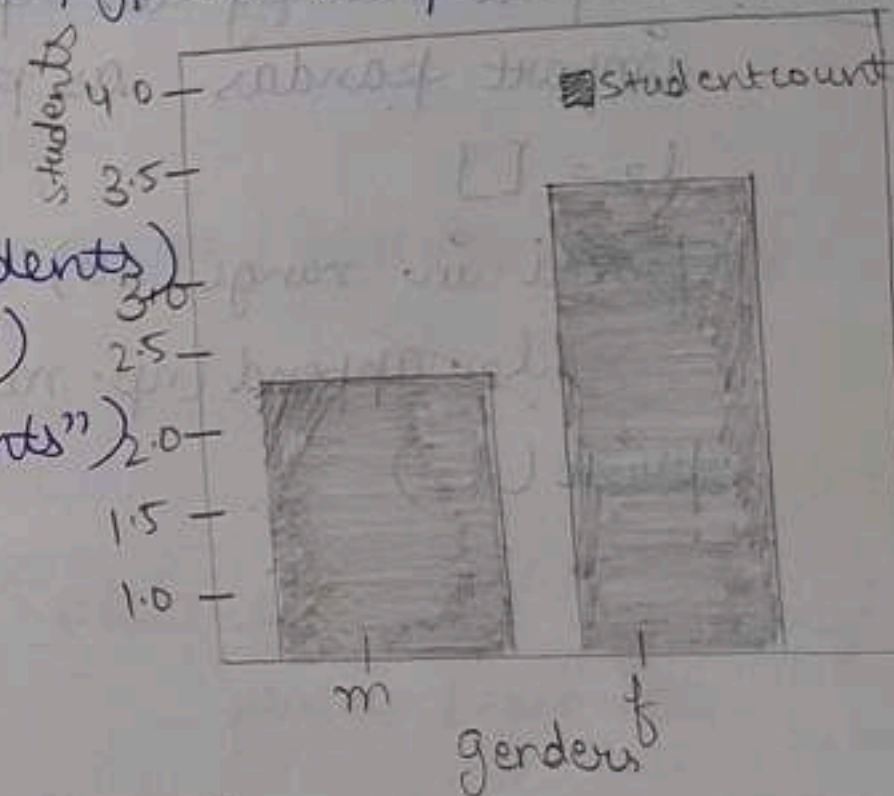
students = (23, 34)

plt.bar(gender, students)

plt.xlabel("genders")

plt.ylabel("students")

plt.show()



marks = [10, 20, 30]

plt.hist(marks)

plt.title("Students data")

plt.xlabel("marks of students")

plt.legend("marks")

plt.show()

Ans.4 Ex: data = np.random.randint(1,100, size=(1000,3))
print(data)

integer

divide
[1,100]

→ import numpy as np

l1 = []

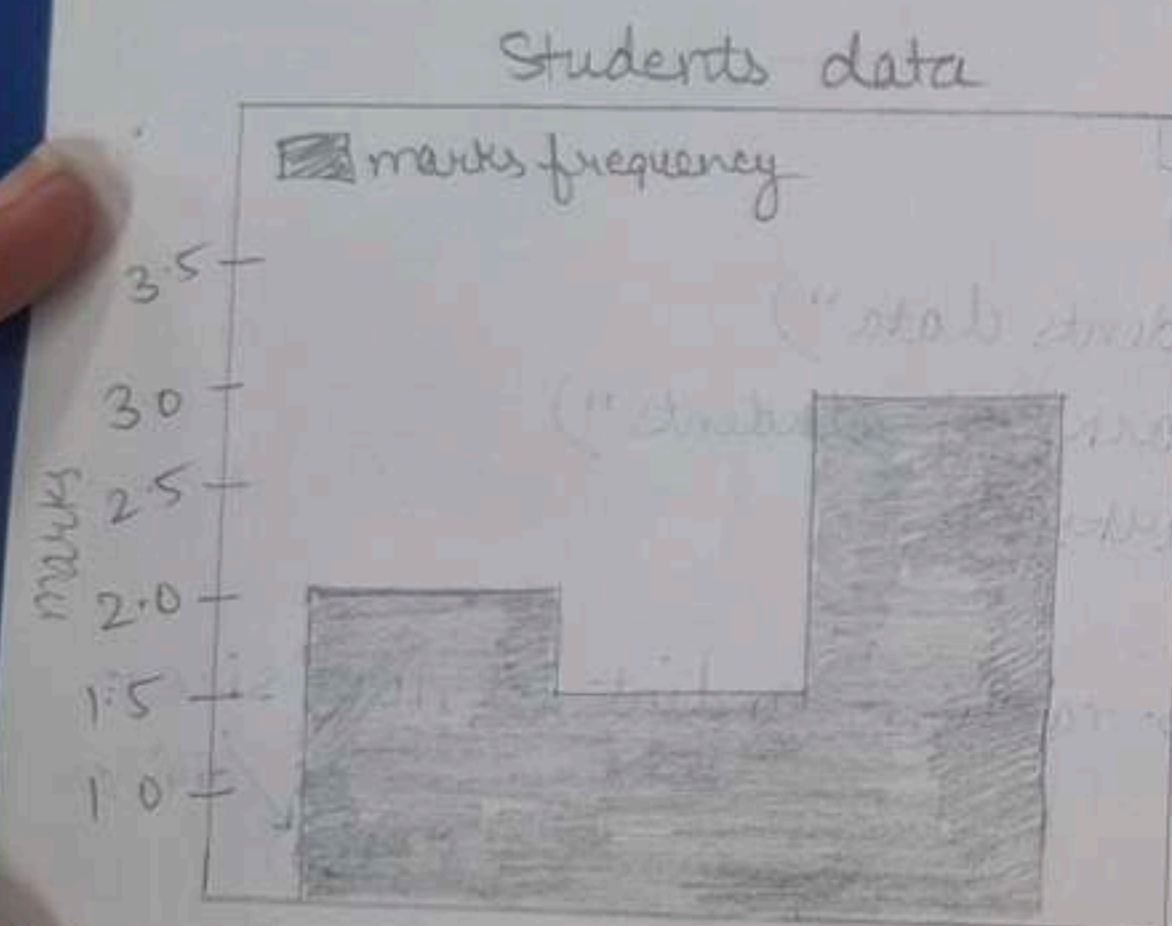
for i in range(30):

l1.append(np.random.randint(0,100))

print(l1)

Ans 5 `data3 = np.random.randn(1,5)`
`print(data3)`
`plt.hist(data3)`
`plt.show()`

Ans 6 `import numpy as np`
`import pandas as pd`
`l2 = []`
`for i in range(10):`
`l2.append(np.random.randn(1,5))`
`print(l2)`



marks of students

19/Jan/2026

Ques: A dataset is provided, load the csv file having the columns record-id, patient-id, diagnosis, bp, sugar-level, visit-cost and other dataset patient-details with patient-id, name, age, gender, city, admission-date, height-cm.

Ans 1) `import pandas as pd`
`x = pd.read_csv("C:/Users/GIEU/Downloads/medical-records.csv")`
`x`

2) `y = pd.read_csv("C:/Users/GIEU/Downloads/patient-details.csv")`
`y`

Output 1

	record-id	patient-id	diagnosis	bp	sugar-level	visit-cost
0	R101	P013	Hypertension	120/80	140.0	2000
1	R102	P105	Diabetes	120/80	NaN	2500
2	R103	P098	Hypertension	high	140.0	3000
3	R104	P038	Asthma	140/90	NaN	3000
4	R105	P040	Asthma	130/85	NaN	1500
...	...	...	...	...	...	...
118	R219	P011	AsHypertension	130/85	180.0	2500
119	R220	P013	Hypertension	high	180.0	1500

120 rows x 6 columns

Output 2

	patient-id	name	age	gender	city	admission	height
0	P001	NaN	25.0	F	Mumbai	15/01/24	180
1	P002	Rahul Sharma	60.0	F	Delhi	24/01/16	170
2	P003	Neha Singh	45.0	M	Mumbai	24/01/16	150
3	P004	"	60.0	Male	Kolkata	15/01/24	170
4	P005	"	25.0	Male	Kolkata	24/01/15	180
...	...	...	...	...	...	...	...
128	P009	John Paul	25.0	F	Mumbai	2024/01/15	150
129	P010	NaN	28.0	M	Chennai	2024/01/16	150

130 rows x 7 columns.

Operations

1) x.head(4)

Output

	record-id	patient-id	diagnosis	bp	sugar-level	visit-cost
0	R101	P013	Hypertension	120/80	140.0	2000
1	R102	P015	Diabetes	120/80	NaN	2500
2	R103	P098	Hypertension	High	140.0	3000
3	R104	P038	Asthma	140/90	NaN	3000

2) x.tail(3)

Output

	record-id	patient-id	diagnosis	bp	sugar-level	visit-cost
117	R218	P063	Hypertension	140/90	NaN	2000
118	R219	P011	"	130/85	180.0	2500
119	R220	P013	"	high	180.0	1500

3) x.info()

Output

```
<class 'pandas.core.frame.DataFrame'>
Range Index: 120 entries, 0 to 119
Data Columns (total 6 columns):
# Column      Non-Null count  Dtype

```

0	record-id	120 non-null	object
1	patient-id	"	"
2	diagnosis	"	"
3	bp	"	"
4	sugar-level	85 non-null	float64
5	visit-cost	120 non-null	int64

dtypes: float64(1), int64(1), object(4)
memory usage: 5.8 + KB

4. x.describe()

	sugar-level	visit-cost
count	85.000000	120.000000
mean	141.058824	2287.500000
std	27.689758	567.548539
min	110.000000	1500.000000
25%	110.000000	2000.000000
50%	140.000000	2500.000000
75%	180.000000	3000.000000
max	180.000000	"

5. x.isnull().sum()

Output

record-id	0
patient-id	0
diagnosis	0
bp	0
sugar-level	35
visit-cost	0
dtype:	int64

6. x.isnull()

	record-id	patient-id	diagnosis	bp	sugar-level	visit-cost
0	False	False	False	False	False	False
1	False	False	False	False	True	False

7. x.shape
(120, 6)

8. x.size
720

9. x.isna()

	record-id	patient-id	diagnosis	bp	sugar-level	visit-cost
0	false	false	false	false	false	false
1	false	false	false	false	false	true

10. x.isna().sum()

record-id	0
patient-id	0
diagnosis	0
bp	0
sugar-level	35
visit-cost	0
dtype: int64	

11. x["sugar-level"].fillna("marwi")

0	140.0
1	marwi
2	140.0
3	marwi
4	marwi

118	180.0
119	180.0

12. yx = y["age"].mean()

Output

np.float(35.84905660377358)

13. x["record-id"].isna().sum()

→ np.int64(0)

2/ Feb/26

Ques. 1 Implement dfs code.

Ans.

```
def dfs(st, goal, graph, vis = None):
```

```
    if vis is None:
        vis = set()
```

```
    print(st)
```

```
    if st == goal:
```

```
        return True
```

```
    vis.add(st)
```

```
    for child in graph[st]:
```

```
        if child not in vis:
```

```
            if dfs(child, goal, graph, vis):
```

```
                return True
```

```
    return False
```

```
graph = {'a': ['b', 'c'],
```

```
        'b': ['a', 'c'],
```

```
        'c': ['a', 'b', 'd'],
```

```
        'd': ['c', 'e'],
```

```
        'e': ['d']}
```

```
a = dfs('a', 'd', graph)
```

```
a
```

Output

a

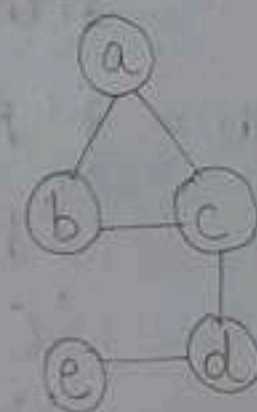
b

c

d

True

Source →
Destination



bfs → a, c, d

Ques 2 Implement bfs code.

Ans.

```
def bfs(st, goal, graph):
    vis = set()
    queue = [st]
    path = []
    vis.add(st)
    while queue:
        node = queue.pop(0)
        path.append(node)
        if node == goal:
            return path
        for neighbour in graph[node]:
            if neighbour not in vis:
                vis.add(neighbour)
                queue.append(neighbour)
    return path
```

```
graph = {'A': ['B', 'C'],
        'B': ['D'],
        'C': ['E'],
        'D': [],
        'E': []}
```

bfs('A', 'D', graph)

Output

['A', 'B', 'C', 'D']



→ bfs('A', 'E', graph)

Output

['A', 'B', 'C', 'D', 'E']

9/2/24

Ques 1 Perform Min-Max Normalization on the following dataset.

Student Data → Student Roll.No., Name, Height, Weight, Stipend

Min 50 data columns (rows)

Ques 2 What is a graph and why do we use it?

Ques 3 Types of graph.

Ques 4 What is bfs and why do we use bfs? Real life applications of bfs (5)

Ques 5 What is dfs and why do we use it? Real life applications of dfs (5)

Ans 1 Import pandas as pd

```
d = pd.read_csv("C:\\Users\\GEU\\Desktop\\P.csv")
d
```

→ Output

	roll.no	name	height	Weight	stipend
0	1	DEEPAK SINGH	111	40	11000
1	2	NITIN RAWAT	120	48	12000
2	3	HRITHIK SINGH RAWAT	113	53	13000
3	4	UTKARSH CHAUDHARY	127	23	14000
...	...	...	...	...	...
46	47	PINKY	172	75	10010
47	48	ABHISHEK PAL	173	76	10020
48	49	MOHAMMAD AHMAD	174	77	10030
49	50	JAHANVI SINGH ANDOTRA	175	78	10040

```
c = ["height", "weight", "stipend"]
```

for x in c:

```
d[x] = (d[x] - d[x].min()) / (d[x].max() - d[x].min())
```

```
d.to_csv("pdata.csv", index=False)
```

roll no.	name	height	weight	stipend
2	1 DEEPAK SINGH	0	0.375	0.133
3	2 NITIN	0.115385	0.475	0.155
4	3 HRITHIK	0.025641	0.5375	0.177
5	4 UTKARSH	0.205128	0.1625	0.2
...	...	...	...	...
48	47 PINKY	0.782	0.8125	0.1113
49	48 ABHISHEK	0.7948	0.825	0.11155
50	49 MOHAMMAD	0.80769	0.8375	0.11177
51	50 JAHANVI	0.8205	0.85	0.112

Ans 2 Graph: A graph is a visual representation of data, consisting of nodes (vertices) connected by lines (edges) to show relationships between variables, on a Cartesian plane.

→ We use graphs to simplify complex data, identify trends, compare values and model networks, making information easier to understand and analyze.

Ans 3 Types of Graphs:

1. Bar graphs → for comparing categories
2. Line graphs → for trends over time
3. Pie charts → for proportions
4. Histograms → for frequency distributions
5. Scatter plots → for relationships
6. Box plot → to identify outliers.

Ans 4 Breadth-First Search (BFS): It is a fundamental graph traversal algorithm that explores nodes level by level, visiting all neighbors of a node before moving to the next level, usually employing a queue data structure.

→ We use bfs to find the shortest path in unweighted graphs, detect cycles, and traverse data structures efficiently.

→ Shortest path Calculation, Cycle-Detection.

Real life Applications

1. GPS Navigation Systems
2. Social Networking Sites
3. Network Broadcasting
4. AI and Game Development
5. Garbage Collection in Programming.

Ans 5 Depth-First Search (DFS): It is a fundamental graph traversal algorithm that explores as deep as possible along each branch before backtracking, utilizing a stack (often recursion) to manage nodes.

→ It is used for traversing trees/graphs, detecting cycles, topological sorting and solving pathfinding puzzles.

→ Memory Efficiency, Pathfinding in Puzzles, Cycle Detection, Topological Sorting.

Real life Applications

1. Web Crawlers
2. File System Navigation
3. AI Game solving
4. Puzzle Solving
5. Social Networking → Finding connected groups of friends

Ques 1 Design and Develop a program to implement the dfs algorithm for graph reversal in artificial intelligence in the following specifications.

- i) Construct a static graph consisting of 6 nodes and 10 edges where the graph is stored using a dictionary representation, the graph is given as: a-b, a-c, b-d, b-e, c-d, c-f, d-e, d-f, e-f, b-f.

ii) Create the graph where the no. of nodes and edges are provided by the user at the run time.

iii) Accept the initial and the goal node from the user.

iv) Display the path from the initial node to the goal node for both the cases.

Ans. i) def dfs(st, goal, graph, vis = None):

if vis is None:

vis = set()

print(st)

if st == goal:

return True

vis.add(st)

for child in graph[st]:

if child not in vis:

if dfs(child, goal, graph, vis):

return True

return False

graph = {'a': ['b', 'c'], 'b': ['d', 'e', 'f'], 'c': ['d', 'f'], 'd': ['e', 'f'], 'e': ['f'], 'f': []}

a = dfs('a', 'c', graph)

a

→ a

b

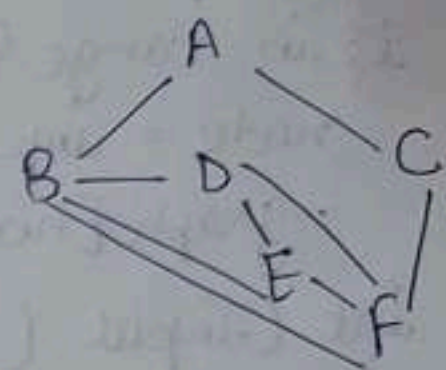
d

e

f

c

True



Ques 2 Design and develop a program to implement bfs algorithm for graph traversal in ai with the following specifications.

- i) Construct a static graph consisting of 6 nodes and 10 edges where the graph is stored using a dictionary representation. The graph is given as

a-b, a-c, b-d, b-e, c-d, c-f, d-e, d-f, e-f, b-f

- ii) Create another graph dynamically where the no. of nodes and edges are given by the user at runtime and the user specifies which nodes are connected. iii) Accept the initial node, goal node from the user. iv) Perform the bfs from the given starting node and goal node. v) Display the order of node traversal. vi) Display the shortest path for both static and dynamic cases.

```

ii) graph = {}
n = int(input("Enter no. of nodes"))
for i in range(n):
    node = input("Enter node name")
    graph[node] = []
e = int(input("Enter no. of edges"))
for i in range(e):
    u = input("Enter starting node")
    v = input("Enter ending node")
    graph[u].append(v)
    graph[v].append(u)
for Key in graph:
    print(Key, ': ', graph[Key])

```

Output

```

Enter no. of nodes 6
Enter node name a
Enter node name b
Enter node name c
Enter node name d
Enter node name e
Enter node name f
Enter no. of edges 10
Enter starting node a
Enter ending node b
" starting " a
" ending " c

```

```

Enter starting node b
" ending node d
" starting " b
" ending " e
" starting " c
" ending node d
" starting " c
" ending " f
" starting " d
" ending " e
" starting node d
" ending node f
" starting " e
" ending " f
" starting node b
" ending node f

```

```

a: ['b', 'c']
b: ['a', 'd', 'e', 'f']
c: ['a', 'd', 'f']
d: ['b', 'c', 'e', 'f']
e: ['b', 'd', 'f']
f: ['c', 'd', 'e', 'b']

```

Code `x = dfs('a', 'c', graph)`

Output

```

a
b
d
e
f
True

```

```

iii) initial = input("Enter start node")
final = input("Enter final node")
x = dfs(initial, final, graph)
x

```

Output

```

Enter start node a
Enter final node c
a
b
c
d
e
f

```

True

Ans 2 i) Static case

```

def bfs(st, goal, graph):
    vis = set()
    queue = [st]
    path = []
    vis.add(st)
    while queue:
        node = queue.pop(0)
        path.append(node)
        if node == goal:
            return path
        for neighbor in graph[node]:
            if neighbor not in vis:
                vis.add(neighbor)
                queue.append(neighbor)

```

return path

```

graph = {'a': ['b', 'c'], 'b': ['d', 'e', 'f'],
        'c': ['d', 'f'], 'd': ['e', 'f'], 'e': ['f'],
        'f': []}

```

```

x = bfs('a', 'c', graph)

```

Output ['a', 'b', 'c']

ii) Dynamic

```

graph = {}

```

```

n = int(input("Enter no. of nodes"))

```

```

for i in range(n):

```

```

    node = input("Enter node name")

```

```

    graph[node] = []

```

```

e = int(input("Enter no. of edges"))

```

```

for i in range(e):

```

```

    u = input("Enter starting node")

```

```

    v = input("Enter ending node")

```

```

    graph[u].append(v)

```

```

    graph[v].append(u)

```

```

for key in graph:

```

```

    print(key, ': ', graph[key])

```

Output

```

Enter no of nodes 6

```

```

Enter node name a

```

```

" " " b

```

```

" " " c

```

```

" " " d

```

```

" " " e

```

Enter node name f
 Enter no. of edges 10
 Enter starting node a
 " ending " b
 " starting " a
 " ending " c
 " starting " b
 " ending " d
 " starting " b
 " ending " e
 " starting " c
 " ending " d
 " starting " c
 " ending " f
 " starting " d
 " ending " e
 " starting " d
 " ending " f
 " starting " b
 " ending " f

a: ['b', 'c']
 b: ['a', 'd', 'e', 'f']
 c: ['a', 'd', 'f']
 d: ['b', 'c', 'e', 'f']
 e: ['b', 'd', 'f']
 f: ['c', 'd', 'e', 'b']

Code
 x1 = bfs('a', 'c', graph1)
 x1

Output ['a', 'b', 'c']

iii) initial = input("Enter start node")
 final = input("Enter final node")
 x2 = bfs(initial, final, graph1)

x2
 Output
 → ['a', 'b', 'c']

x3 = bfs(initial, final, graph)

x3
 Output
 → ['a', 'b', 'c']

Ques: Code for applying Standard scalar to a list named data.

Ans:

```
def standard(data):
    mean = sum(data) / len(data)
    sumsq = 0
    for x in data:
        sumsq += (x - mean) ** 2
    std = (sumsq / len(data)) ** 0.5
    scaled = []
    for x in data:
        scaled.append((x - mean) / std)
    return scaled
```

```
data = [10, 20, 30, 40, 50]
print(standard(data))
```

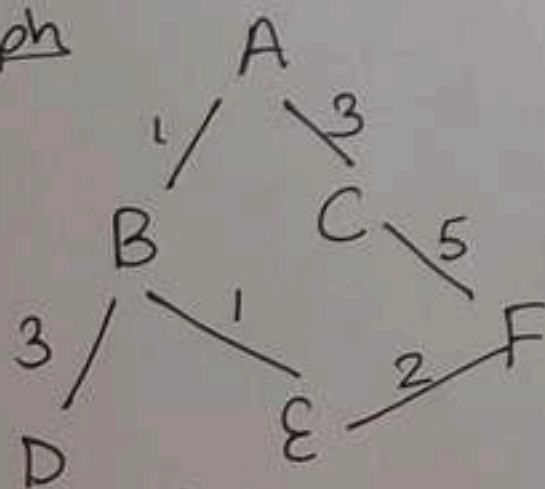
Output

$[-1.414, -0.707, 0.0, 0.707, 1.414]$

Ques: Write a code to implement astar to a graph with given heuristic values as:

A = 6, B = 4, C = 4, D = 2, E = 2, F = 0

Graph



Ans:

```
def astar(start, goal, graph, h):
    open = [start]
    g = {start: 0}
    parent = {start: None}
    while open:
        n = open[0]
        for i in open:
            if g[i] + h[i] < g[n] + h[n]:
                n = i
        if n == goal:
            path = []
            while n:
                path.append(n)
                n = parent[n]
            return path[::-1]
        open.remove(n)
        for m in graph[n]:
            if m not in g:
                g[m] = g[n] + graph[n][m]
                parent[m] = n
                open.append(m)
```

```
graph = {'A': {'B': 1, 'C': 3},
        'B': {'D': 3, 'E': 1},
        'C': {'E': 5, 'F': 2},
        'D': {},
        'E': {'F': 2},
        'F': {}}
```

```
h = {'A': 6, 'B': 4, 'C': 4, 'D': 2, 'E': 2, 'F': 0}
```

```
print(astar('A', 'F', graph, h))
```

Output

['A', 'B', 'E', 'F']

Ques IQR method code to identify outliers in a given series of data.

Ans:

```
import pandas as pd
data = pd.Series([10, 12, 14, 15, 18, 20, 100])
q1 = data.quantile(0.25)
q3 = data.quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outliers = data[(data < lower) | (data > upper)]
print(f"Outliers : {outliers.tolist()}")
```

Output

Outliers : [100.0]

16/3/24

Ques 1. Applying Linear Regression model on a dataset and calculating MSE, MAE, RMSE, r^2 Score and plotting the graph of the Best fit line.

Step 1

- import numpy as np
- import pandas as pd
- from sklearn.linear_model import LinearRegression
- from sklearn.model_selection import train_test_split
- from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
- import matplotlib.pyplot as plt

Step 2 data = {"x": [1, 2, 3, 4, 5, 6, 7, 8], "y": [2, 4, 6, 8, 10, 12, 14, 16]}

df = pd.DataFrame(data)

df

Output

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10
5	6	12
6	7	14
7	8	16

Step 3 X = df[['x']]
Y = df['y']

Step 4 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

Steps
 X_train, X_test, Y_train, Y_test

Output

0 10
 7 8
 2 3
 4 5
 3 4
 6 7
 1 2
 5 6
 0 2
 7 16
 2 6
 4 10
 3 8
 6 14
 Name: y, dtype: int64
 1 4
 5 12
 Name: y, dtype: int64

Step 6 model = LinearRegression()

Step 7 model.fit(X_train, Y_train)

Output

LinearRegression
 Parameters

Step 8 Y_pred = model.predict(X_test)

Step 9 Y_pred

Out: array([4., 12.])

Step 10 Y_test

Out: 1 4
 5 12

Name: y, dtype: int64

Step 11 slope = model.coef[0]
 intercept = model.intercept -
 slope, intercept

Out: (1.999999999...15), 5.329070518200751e-15

Step 12 mean_absolute_error(Y_test, Y_pred)

Out: 2.6645352591003757e-15

Step 13 mean_squared_error(Y_test, Y_pred)

Out: 7.888609052210118e-30

Step 14 np.sqrt(mean_squared_error(Y_test, Y_pred))

Out: 2.808666774861361e-15

Step 15 r2_score(Y_test, Y_pred) Out: 1.0

Step 16

plt.scatter(X, Y, color="blue", label="Actual Data")

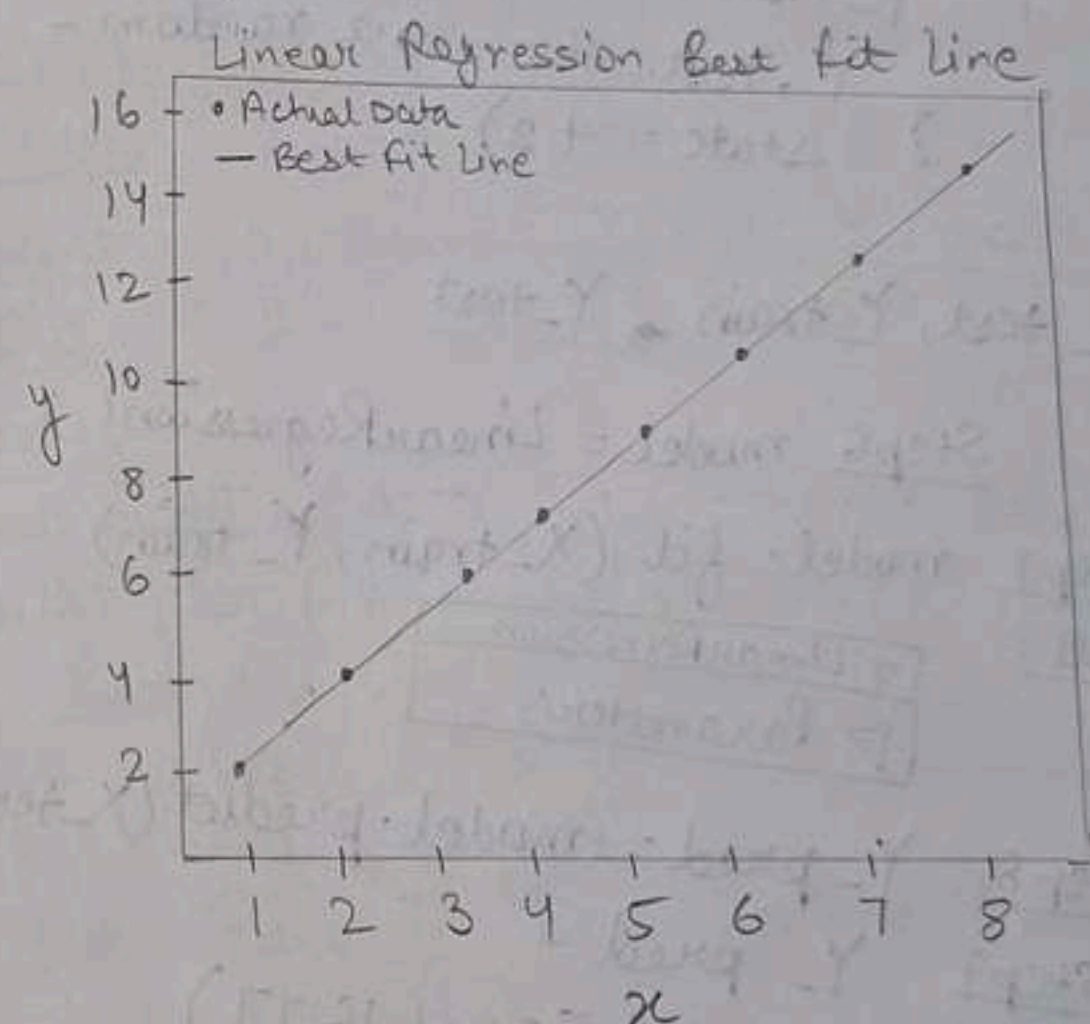
plt.plot(X, model.predict(X), color="red", label="Best fit Line")

plt.xlabel("x")

plt.ylabel("y")

plt.title("Linear Regression Best fit Line")

plt.legend()



Ques 2 Applying Linear Regression model on a dataset and calculating MSE, RMSE, MAE, r2 score and plotting the Graph of the Best fit line.

Step 1 import numpy as np
 import pandas as pd
 import matplotlib.pyplot as plt
 from sklearn.linear_model import LinearRegression
 from sklearn.model_selection import train_test_split
 from sklearn.metrics import mean_squared_error,
 mean_absolute_error, r2_score

Step2 data = {"x": [11, 12, 13, 14, 15, 16, 17, 18],
 "y": [22, 23, 33, 11, 12, 14, 16, 17]}

df = pd.DataFrame(data)

Output:

	x	y
0	11	22
1	12	23
2	13	33
3	14	11
4	15	12
5	16	14
6	17	16
7	18	17

Step3 X = df[['x']]
 Y = df['y']

Step4 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2, random_state = 42)

Step5 X_train, X_test, Y_train, Y_test

Output:

	x	y
0	11	22
1	12	23
2	13	33
3	14	11
4	15	12
5	16	14
6	17	16
7	18	17

Step6 model = LinearRegression()

Step7 model.fit(X_train, Y_train)

Out:

LinearRegression
 Parameters

Step8 Y_pred = model.predict(X_test)

Step9 Y_pred

Out: array([22.1, 16.7])

Step10 Y_test

Output:

1	23
5	14

Name: y, dtype: int64

Name: y, dtype: int64

1 23
 5 14

Name: y, dtype: int64

Step11 slope = model.coef_[0]
 intercept = model.intercept_
 slope, intercept

Out: (-1.3499999999999999, 38.3)

Step12 mean_absolute_error(Y_test, Y_pred)

Out: 1.8000000000000007

Step13 mean_squared_error(Y_test, Y_pred)

Out: 4.05

Step14 np.sqrt(mean_squared_error(Y_test, Y_pred))

Out: 2.0124611797498106

Step15 r2_score(Y_test, Y_pred)

Out: 0.8

Step16

plt.scatter(X, Y, color = "blue", label = "Actual Data")

plt.plot(X, model.predict(X), color = "red", label = "Best fit line")

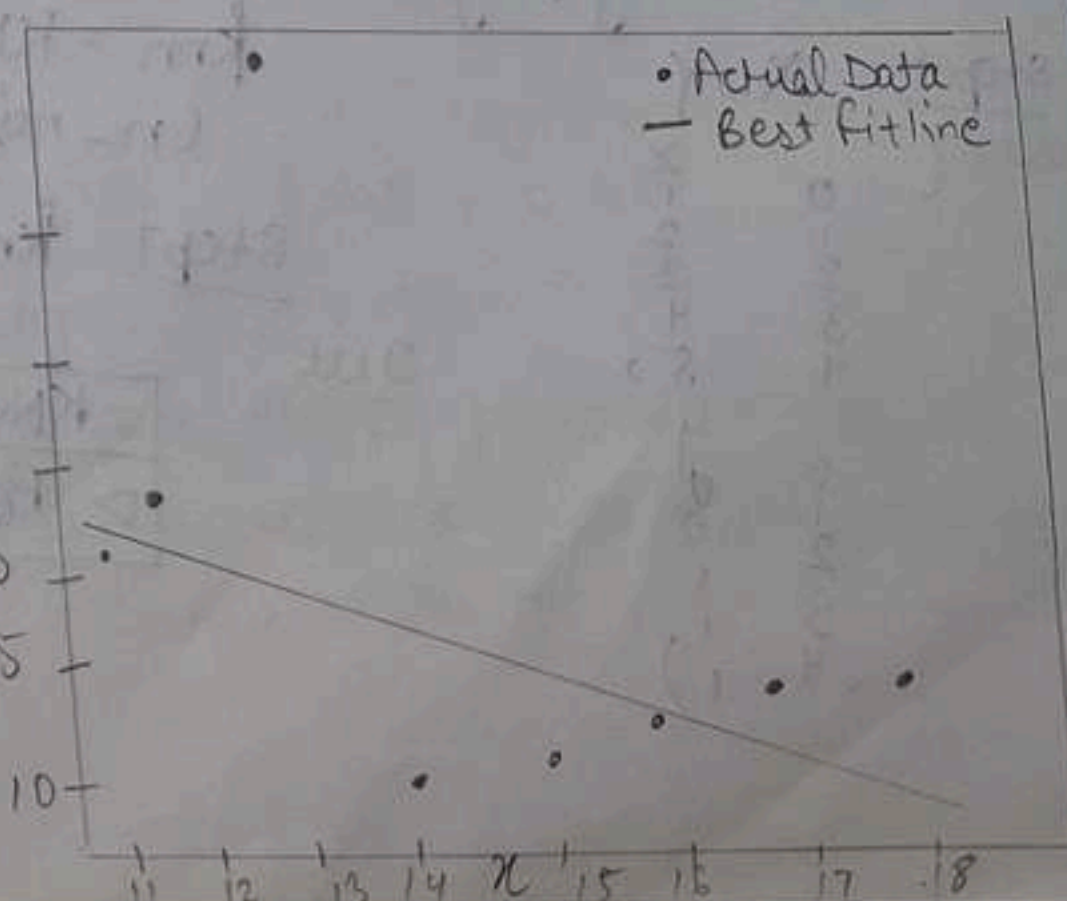
plt.xlabel("x")

plt.ylabel("y")

plt.title("Best fit line Graph")

plt.legend()

Out:



Ques 3 Apply Linear Regression model on dataset and calculate accuracy-score, precision-score, recall-score, f1-score, confusion-matrix

Step 1. import numpy as np
 from sklearn.neighbors import KNeighborsClassifier
 from sklearn.model_selection import train-test-split
 import pandas as pd
 from sklearn.metrics import accuracy-score, precision-score, recall-score, confusion-matrix, classification-report, f1-score

Step 2 data = {"x": [1, 2, 3, 4, 5], "y": [0, 0, 1, 1, 1]}
 df = pd.DataFrame(data)

Output:

	x	y
0	1	0
1	2	0
2	3	1
3	4	1
4	5	1

Step 3 X = df[['x']]
 Y = df[['y']]

Step 4 Out: (

	x	y
0	1	0
1	2	0
2	3	1
3	4	1
4	5	1

Step 5 X_train, X_test, Y_train, Y_test = train-test-split(X, Y, test-size=0.2, random-state=42)

Step 6 knn = KNeighborsClassifier(n_neighbors=3)

Step 7 knn.fit(X_train, Y_train)

Out:

▽ KNeighborsClassifier
▷ Parameters

Step 8 Y_pred = knn.predict(X_test)

Step 9 Y_test

	y
0	0
1	0
2	1
3	1
4	1

Step 10 Y_pred

Out: array([1])

Step 11 accuracy_score(Y_test, Y_pred)

Out: 0.0

Step 12 precision_score(Y_test, Y_pred)

Out: 0.0

Step 13 recall_score(Y_test, Y_pred)

Out: 0.0

Step 14 print(confusion_matrix(Y_test, Y_pred))

Out: [[0 1]
[0 0]]

Step 15 print(classification_report(Y_test, Y_pred))

Out:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	1.0
1	0.00	0.00	0.00	0.0
accuracy			0.00	1.0
macro avg	0.00	0.00	0.00	1.0
weighted avg	0.00	0.00	0.00	1.0

Ques 4 Calculate accuracy-score, precision-score, recall-score, confusion matrix, classification-report from a given dataset.

Step 2

```
data = {"x": [1, 2, 3, 4, 5, 6, 7, 8], "y": [2, 2, 3, 2, 2, 2, 3, 3]}
```

```
df = pd.DataFrame(data)
```

Out:

	x	y
0	1	2
1	2	2
2	3	3
3	4	2
4	5	2
5	6	2
6	7	3
7	8	3

```
Step 1 import numpy as np
import pandas as pd

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report
```

```
Step 3 X = df[['x']]
Y = df[['y']]
X, Y
```

Out:

	x
0	1
1	2
2	3
3	4
4	5
5	6
6	7
7	8

```
Step 5 X_train, X_test,
Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
```

```
Step 6 knn = KNeighborsClassifier(n_neighbors=3)
```

```
Step 7 knn.fit(X_train, Y_train)
```

Out: KNeighborsClassifier
Parameters

```
Step 8 Y_pred = knn.predict(X_test)
```

```
Step 9 Y_test
```

Out:

	y
1	2
5	2

```
Step 10 Y_pred
```

Out: array([2, 2])

```
Step 11 accuracy_score(Y_test, Y_pred)
```

Out: 1.0

```
Step 12 precision_score(Y_test, Y_pred)
```

Out: 0.0

```
Step 13 recall_score(Y_test, Y_pred)
```

Out: 0.0

```
Step 14 print(confusion_matrix(Y_test, Y_pred))
```

Out: [[2]]

```
Step 15 print(classification_report(Y_test, Y_pred))
```

Out:

	precision	recall	f1-score	support
2	1.00	1.00	1.00	2
accuracy	1.00	1.00	1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

23/3/22
Ques. Scaling the data features to a fixed numerical range using MinMax and Standard scaler and calculating mae.

Sol.
Step 1 import numpy as np
 import pandas as pd
 import matplotlib.pyplot as plt
 from sklearn.linear_model import LinearRegression
 from sklearn.model_selection import train_test_split
 from sklearn.metrics import mean_absolute_error
 from sklearn.preprocessing import StandardScaler, MinMaxScaler

Step 2 data = {"x": [11, 12, 13, 14, 15, 16, 17, 18],
 "y": [22, 23, 33, 11, 12, 14, 16, 17]}
 df = pd.DataFrame(data)

Out:

	x	y
0	11	22
1	12	23
2	13	33
3	14	11
4	15	12
5	16	14
6	17	16
7	18	17

Step 3
 X = df[['x']]
 Y = df[['y']]

Step 4 X_train, X_test, Y_train, Y_test =
 train_test_split, test_size = 0.2, random_state
 = 42)

Step 5 X_train, X_test, Y_train, Y_test

Step 6
 scaler = StandardScaler()
 X_train = scaler.fit_transform(X_train)
 X_test = scaler.fit_transform(X_test)

Step 7 scaler = MinMaxScaler()
 X_train = scaler.fit_transform(X_train)
 X_test = scaler.fit_transform(X_test)

Step 8 model = LinearRegression()
Step 9 model.fit(X_train, Y_train)

~~Y_pred = model.predict(X_train)~~
 Y_pred = model.predict(X_test)

Step 12 Y_test

Step 13 slope = model.coef_[0]
 intercept = model.intercept_
 slope, intercept

Out: (-9.45000001, 23.45000003)

Step 14 mean_absolute_error(Y_test, Y_pred)

Ques ^{6/4/25} Apply Logistic Regression on a dataset
 sol.

Step 1 import pandas as pd
 import numpy as np
 from sklearn.linear_model import LogisticRegression
 from sklearn.model_selection import train_test_split
 from sklearn.metrics import accuracy_score,
 f1_score, confusion_matrix, classification_report

Step 2
 data = {"Studyhours": [1, 2, 3, 4, 5, 6, 7, 8],
 "Result": [0, 0, 0, 0, 1, 1, 1, 1]}
 df = pd.DataFrame(data)

Output

	Studyhours	Result
0	1	0
1	2	0
2	3	0
3	4	0
4	5	1
5	6	1
6	7	1
7	8	1

Step 3 X = df[['Studyhours']]
 Y = df['Result']

Step 4 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.3, random_state=42)

Step 5 X_train, X_test, Y_train, Y_test

Step 6 model = LogisticRegression()

Step 7 model.fit(X_train, Y_train)

Step 8 Y_pred = model.predict(X_test)

Step 9 Y_pred
Output array([0, 1, 0])

Step 10 Y_test

Output
 1 0
 5 1
 0 0

Name: result, dtype: int64

Step 11 accuracy_score(Y_test, Y_pred)

Output
 1.0

Step 12 f1_score(Y_test, Y_pred)

Output
 1.0

Step 13 ~~sk~~ print(classification_report(Y_test, Y_pred))

Output

	precision	recall	f1 score	support
0	1.00	1.00	1.00	2
1	1.00	1.00	1.00	1
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Step 14 print(confusion_matrix(Y_test, Y_pred))

Output [[2 0
 0 1]]

Ques K Means

Step 1 `import pandas as pd`
`import numpy as np`
`import matplotlib.pyplot as plt`
`from sklearn.cluster import KMeans`

Step 2 `data = {"Value": [1, 2, 3, 10, 11]}`
`df = pd.DataFrame(data)`
`df`

Output

	Value
0	1
1	2
2	3
3	10
4	11

Step 3 `X = df[['Value']].values`
`X`

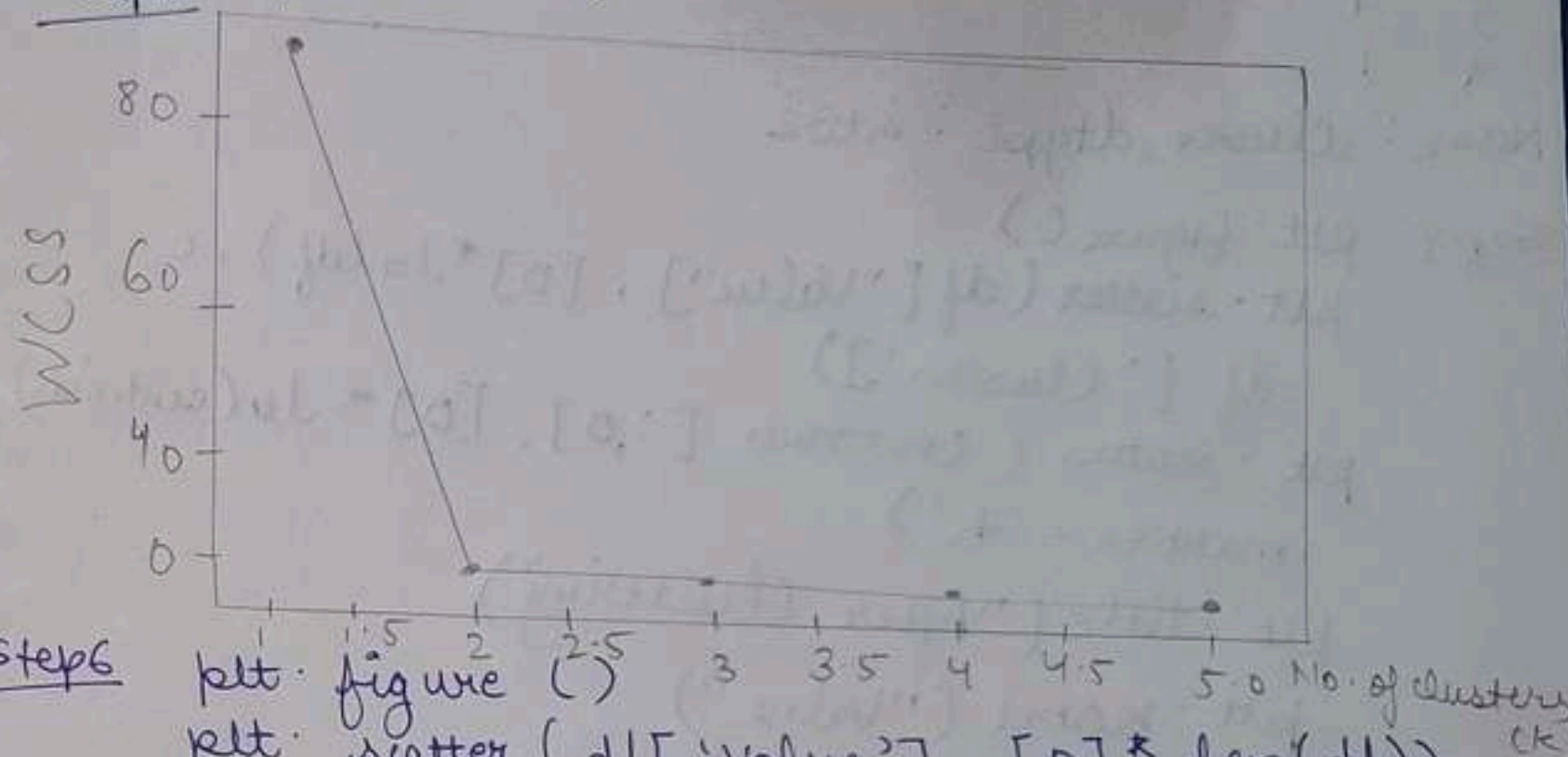
Out: `array([[1],
[2],
[3],
[10],
[11]], dtype=int64)`

Step 4 `wcss = []`

`for k in range(1, 6):`
`kmeans = KMeans(n_clusters=k,`
`random_state=42)`
`kmeans.fit(X)`
`wcss.append(kmeans.inertia_)`

Steps `plt.plot(range(1, 6), wcss, marker="o")`
`plt.xlabel("No. of clusters")`
`plt.ylabel("WCSS")`
`plt.title("Elbow method")`
`plt.show()`

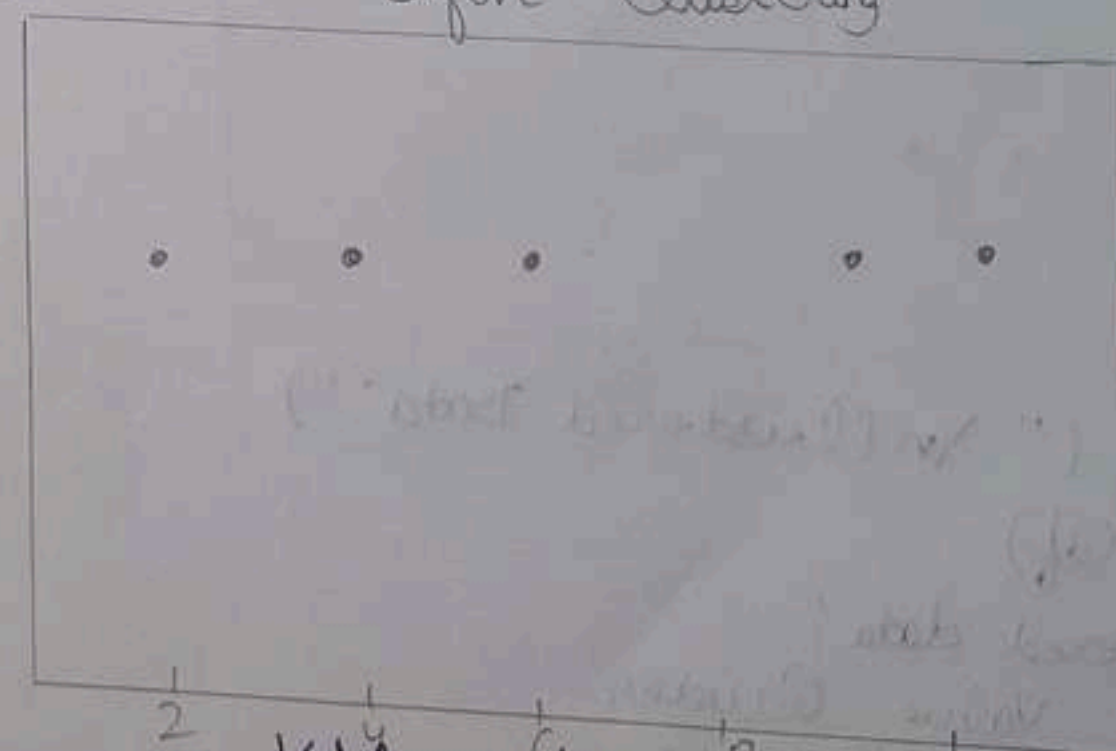
Output



Step 6

`plt.figure()`
`plt.scatter(df['Value'], [0] * len(df))`
`plt.title("Before clustering")`
`plt.xlabel("Value")`
`plt.yticks([])`
`plt.show()`

Output



Step 7

`kmeans = KMeans(n_clusters=2, random_state=42)`
`df['Cluster'] = kmeans.fit_predict(X)`
`centroids = kmeans.cluster_centers_`

Step 8 df['cluster']

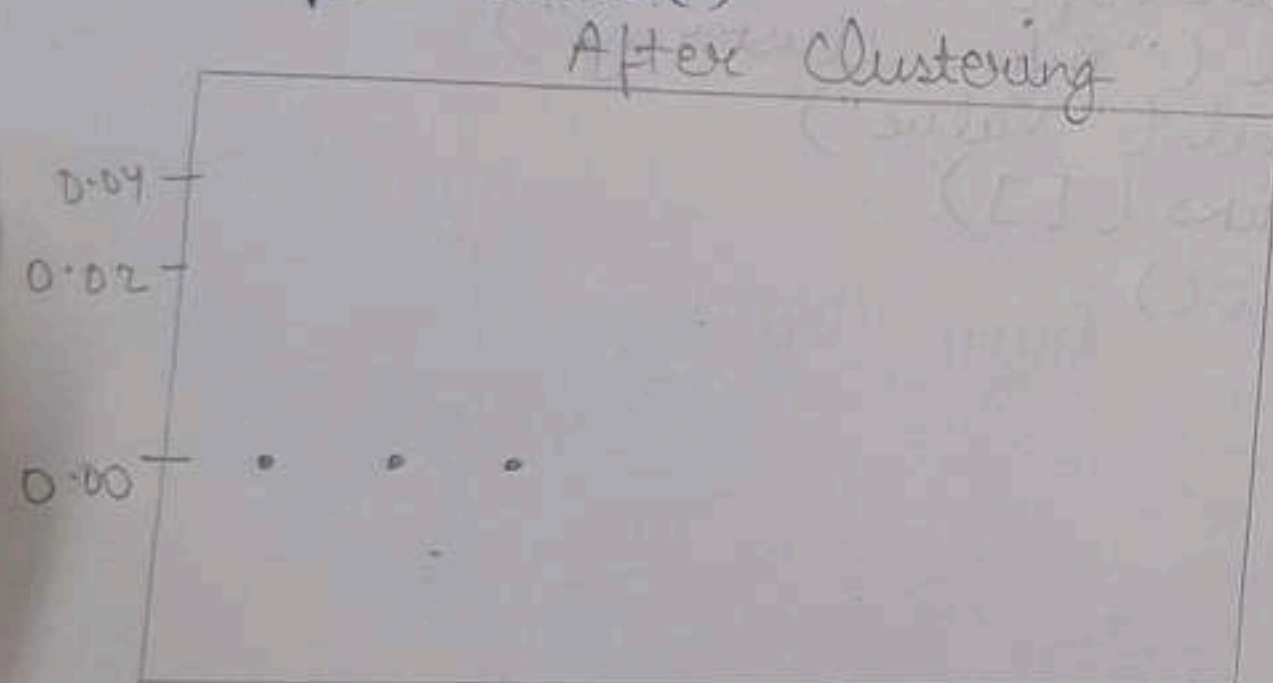
Output

```
0 0
1 0
2 0
3 1
4 1
```

Name: cluster, dtype: int32

Step 9

```
plt.figure()
plt.scatter(df["Value"], [0]*len(df), c=
            df['cluster'])
plt.scatter(centroids[:,0], [0]*len(centroids),
            marker='h')
plt.title("After Clustering")
plt.xlabel("Value")
plt.show()
```



Step 10

```
print("\nClustered Data:")
print(df)
```

Output

Clustered data:

	Value	cluster
0	1	0
1	2	0
2	3	0
3	10	1
4	11	1